需求工程与架构驱动分析

从用户、场景和质量属性出发，把“想要什么”转化为“能够设计和验证什么”。

本讲核心问题

需求为什么会说不清、变不停、做了不对？

从问题进入工程

本讲产出

Persona、核心场景、QAS 清单、变更判断

形成课程产出

课程衔接

承接项目治理，进入“到底做什么”的需求层

连接后续建模

课堂问题：需求不是功能清单，而是把真实问题翻译成工程决策输入的过程。

关系结构

本讲位置：项目治理之后，架构建模之前

可行性回答“能不能做”，需求工程回答“做什么才算对”。

项目治理

约束、计划、风险

→

需求工程

用户、场景、质量属性

→

UML 建模

用例、对象、交互

→

架构设计

质量属性驱动决策

每个环节都要接收上一环节的工程制品，并为下一环节留下可检查输入。

案例冲突

需求为什么难：自然语言天生有歧义

同一句用户表述，不同角色会听出完全不同的系统。

客户说

给三个孩子做一个秋千

产品经理听到

板子和绳子挂在树上

误解放大

工程师看到

树、绳、承重和安全约束

真正问题

孩子要安全、一起玩、树不能被破坏

案例冲突

需求三困境：说不清、变不停、做了不对

需求问题不是偶发沟通失误，而是软件工程中的结构性困难。

说不清

用户经常描述解决方案，而不是问题本身

表现

变不停

看到原型和演示后，真实想法逐渐暴露

后果

做了不对

功能完成了，但质量属性和约束没有满足

处理

课堂错点 名词解释：需求分析的目标。

概念框架

三类需求：功能、质量属性、约束条件

功能说明系统做什么，质量属性说明做得怎样，约束条件说明边界在哪里。

1

功能需求

用户能完成的业务动作

看业务动作

2

质量属性需求

性能、可用性、安全、可维护、可扩展、易用

看架构压力

3

约束条件

团队、技术栈、部署环境、合规、时间

看设计边界

案例冲突

功能需求最显眼，但不是全部

功能需求回答“用户可以做什么”，不能替代质量与约束分析。

表面可见

功能清单

登录、查看设备、提交预约、审批、通知

看到现象

→ 追问为什么 →

工程追问

隐藏问题

谁能预约？多快响应？数据怎样保护？谁来维护？

进入需求分析

课堂错点 判断题：功能清单完整就等于需求完整。

概念框架

质量属性需求：架构最关心的需求

质量属性需求经常比功能需求更直接地驱动架构选择。

性能

响应时间、吞吐量

可用性

故障恢复、服务连续

安全性

认证、授权、数据保护

可维护性

接手修改、理解成本

可扩展性

规模扩大、模块演进

易用性

学习成本、操作效率

案例冲突

同样是预约系统，质量目标不同，架构就不同

同功能不同质量属性，会导致不同的技术结构和治理方案。

表面可见

学院内部系统

几十到几百用户，重点是可维护、低成本交接

看到现象

→ 追问为什么 →

工程追问

全校共享平台

跨学院、跨校区，重点是安全、可用、扩展和审计

进入需求分析

课堂错点　　案例题：说明质量属性如何影响架构风格。

约束条件：设计空间的硬边界

约束不是需求的附属说明，而是架构选择的边界条件。

技术约束

学校统一认证、校内服务器、数据库选型

组织约束

学生团队开发，学院 IT 后续维护

时间约束

课程项目 10 周内可演示

合规约束

学生信息保护、日志留痕、访问控制

关系结构

需求不是一个人的声音：先识别利益相关者

直接用户只是需求来源之一，运维、监管、付款方也会产生关键需求。

研究生
预约方便、冲突少、反馈快

实验员
审批清楚、管理负担低

指导教师
设备利用率和实验安排可见

需求边界

关键：需求不是投票，而是协调冲突

学院 IT
系统可维护、文档完整

信息中心
安全合规、统一认证

开发团队
范围可控、技术可实现

方法拆解

用户需求要翻译成产品需求

用户表达的是愿望，产品需求要变成可设计、可验证的系统行为。

用户说
我想更快预约

→

追问
快在哪里？查找、审批、冲突反馈还是通知？

→

产品需求
95% 预约提交在 2 秒内返回冲突检测结果

→

架构输入
需要冲突检测规则、缓存策略、接口响应指标

转译路径的关键不是把字段列全，而是让每一步都能产生可检查的工程制品。

需求分析三角：目标用户、使用场景、真实问题

真实需求来自三者的交汇，而不是一句孤立的功能描述。

目标用户
谁在使用系统

三者同时成立
才是有效需求

使用场景
什么时候、什么条件下使用

真实问题
要解决的痛点是什么

不是三选一，而是同时检查

Persona：把“用户”具体成可讨论的人

Persona 不是市场画像，而是让团队围绕具体用户做需求判断的分析工具。

基本信息

姓名、角色、背景

目标

他希望完成什么

痛点

现在卡在哪里

能力与限制

技术熟悉度、时间压力、使用环境

案例冲突

经典案例：VS2005 的三类 Persona

同一个工具面对不同能力层级用户，设计重点会完全不同。

Mort

设计含义

非专业程序员，喜欢简单示例和拖拽

Elvis

设计含义

职业开发者，关注效率、属性和控制

Einstein

设计含义

架构师或技术专家，关注扩展和深层能力

同一个工具面对不同能力层级用户，设计重点会完全不同。

结论 简答题：Persona 如何影响产品设计?

案例冲突

预约系统 Persona：李明

学生 Persona 要把目标、压力和操作环境写清楚。

李明

姓名

李明，研二学生

典型用户不是类别名，而是带着目标、限制和痛点进入系统的人。

目标

尽快预约设备完成实验数据采集

痛点

微信群沟通效率低，冲突信息不透明

限制

经常晚上安排实验，对审批进度敏感

课堂动作　**课堂动作：写出自己课程项目的第一个 Persona。**

案例冲突

预约系统 Persona：王老师

管理者 Persona 往往提出可维护、可追溯和流程清晰的需求。

王老

姓名

王老师，实验员

典型用户不是类别名，而是带着目标、限制和痛点进入系统的人。

目标

减少重复沟通，清楚管理设备使用

痛点

人工登记容易漏、审批记录难追溯

限制

希望操作简单，不愿维护复杂系统

课堂动作　**比较题：李明和王老师的需求有什么冲突？**

方法拆解

场景描述：把 Persona 放进具体时间和动作

场景把“谁要什么”推进到“在什么情况下如何完成”。

角色
谁在使用

→

背景
为什么现在要用

→

触发
什么事件启动流程

→

操作
用户怎样与系统交互

→

期望结果
什么结果才算成功

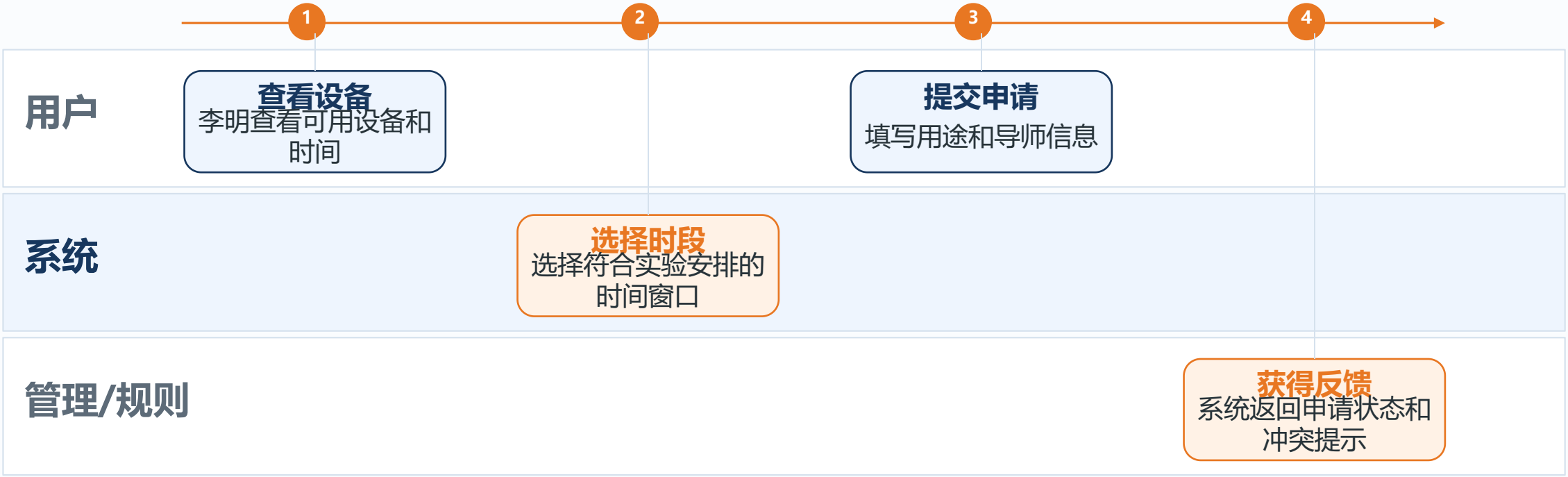
场景脚本的关键不是把字段列全，而是让每一步都能产生可检查的工程制品。

考试锚点 考试可给背景材料，要求写核心使用场景。

案例冲突

核心场景一：正常预约

正常场景用于确认主业务闭环是否成立。

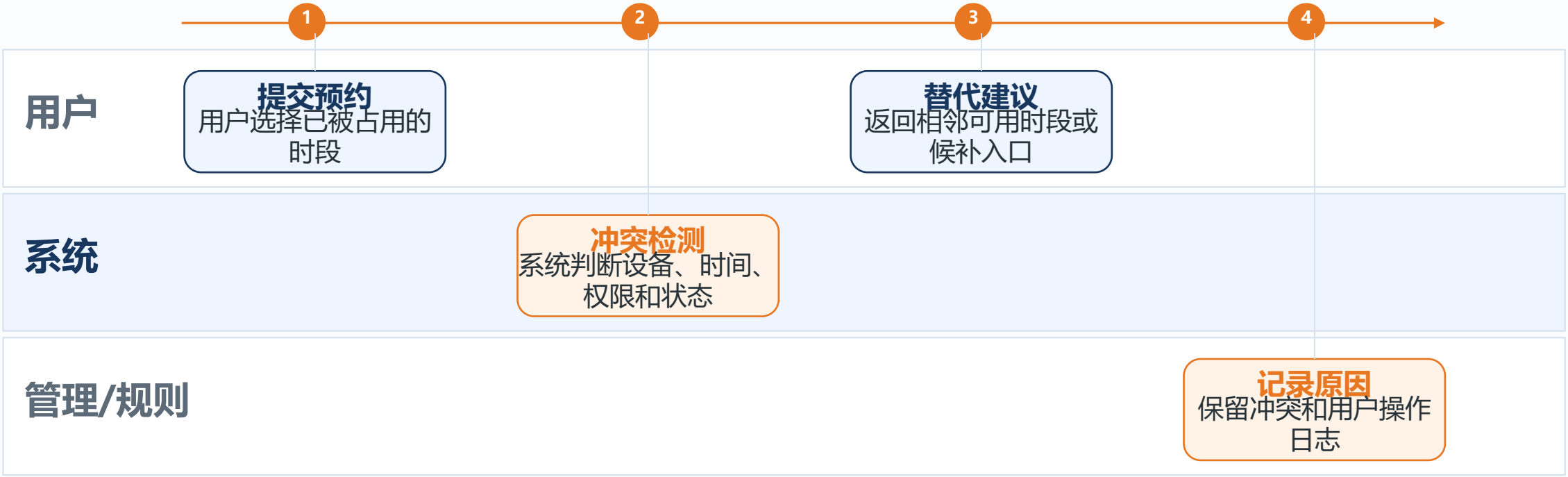


流程图要说明：谁做动作、系统如何响应、规则在哪里生效。

案例冲突

核心场景二：冲突处理

冲突场景往往暴露隐藏质量属性和业务规则。



流程图要说明：谁做动作、系统如何响应、规则在哪里生效。

方法拆解

需求分析三步：挖掘、方案、验证

需求分析不是听写，而是持续追问、提出方案并验证方案。

1. **挖掘真实需求**
   追问表面说法背后的真实问题

→

2. **提出解决方案**
   给出不止一种可行路径

→

3. **筛选与验证**
   用原型、场景和反馈降低误解

方法页要让学生能照着做：先做什么、再产出什么、最后如何验证。

结论　　名词解释：需求分析三步。

案例冲突

从“我要按钮”追到真实问题

用户给出的常常是方案片段，分析者要追问其背后的场景和痛点。

表层说法

我要一个紧急预约按钮

追问一

什么情况下紧急？

误解放大

追问二

谁有权走紧急流程？

真实问题

临时实验窗口需要快速审批且可追溯

课堂错点 课堂练习：把一个模糊需求追问到真实问题。

案例冲突

解决方案不唯一：需求分析也要做工程取舍

同一个真实问题可以有多种方案，方案选择要看成本、风险和约束。

方案	价值	代价	适用判断
方案 A	紧急预约按钮，审批优先级提升	需要额外验证成本	看是否改变系统边界
方案 B	候补队列，资源释放后自动通知	需要额外验证成本	看是否改变系统边界
方案 C	管理员临时授权，事后补充记录	需要额外验证成本	看是否改变系统边界

需求分析不是只找一个方案，而是比较价值、成本和适用边界。

结论　　**简答题：为什么需求分析阶段要比较多种方案？**

案例冲突

原型的目标：快速发现问题，不是做漂亮界面

原型用于验证理解，而不是提前完成产品。

表面可见

误解

原型越精美越好

看到现象

追问
为什么

工程追问

正确理解

越早暴露需求误解越好

进入需求分析

课堂错点　判断题：高保真原型总是优于低保真原型。

方法拆解

三段式原型推进：低、中、高保真

原型保真度应随需求确定性提升，而不是一开始就追求精美。

1. **低保真**
   纸面/白板草图，验证流程结构

→

2. **中保真**
   静态页面或HTML，验证信息组织

→

3. **高保真**
   接近真实交互，验证细节和边界

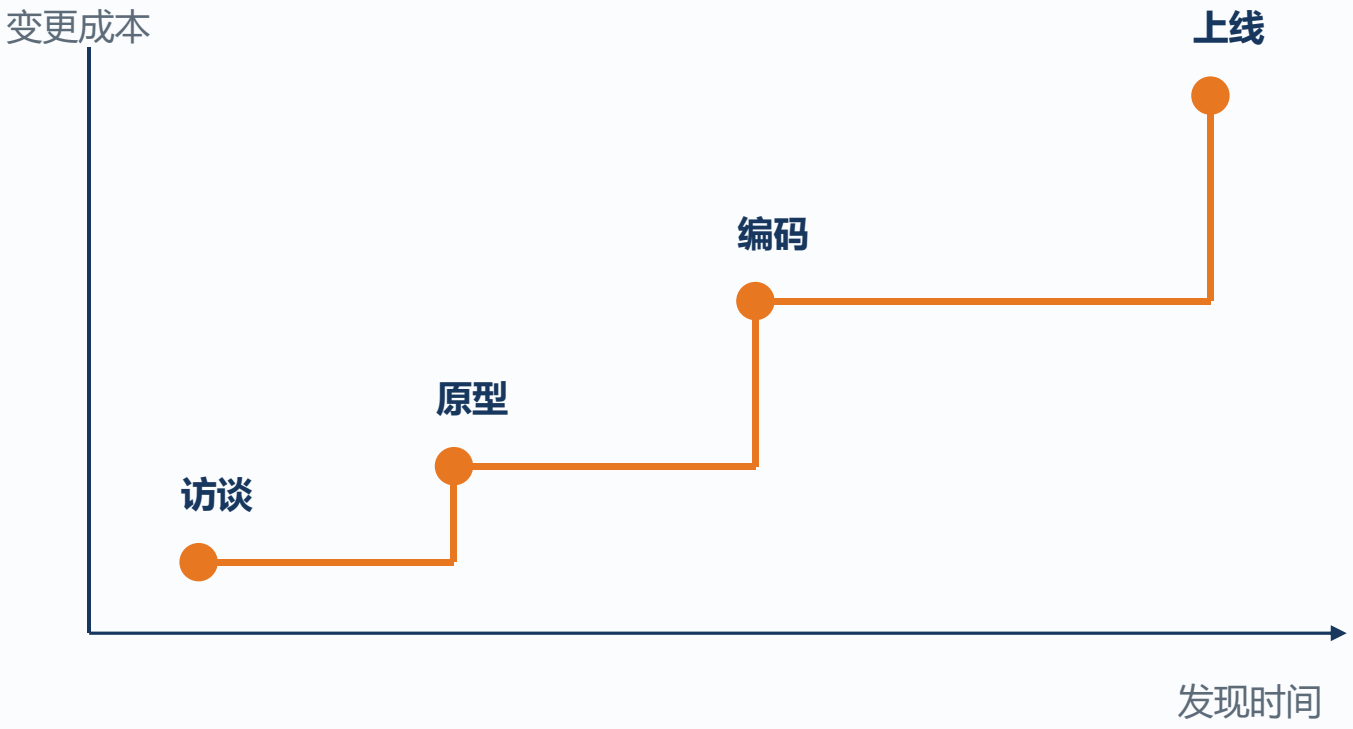
方法页要让学生能照着做：先做什么、再产出什么、最后如何验证。

结论　　简答题：原型设计在需求验证中的价值。

案例冲突

需求变更不是失败信号，但一定有成本

需求变更是正常现象，管理重点是识别代价和影响范围。



早期变更

改场景、改原型，成本低

中期变更

改接口、改数据模型，成本上升

后期变更

改架构、重测、重部署，成本最高

结论 简答题：为什么越晚变更越贵？

案例冲突

骨架变更与皮肤变更要分开处理

不是所有变更都同等重要，关键是判断它是否触动系统骨架。

皮肤变更

文案、颜色、通知细节、统计展示方式

可快速迭代

→

骨架变更

认证方式、数据模型、审批流程、权限边界

必须正式评审

→

处理原则

皮肤快速迭代，骨架正式评审

先判断性质

课堂错点 判断题：给出变更，判断属于骨架还是皮肤。

概念框架

需求文档的三个真实价值

文档不是为了归档，而是为了澄清、协作和未来维护。

1

梳理思路

写不出来，往往说明自己还没想清楚

看业务动作

2

协作沟通

让不同角色对同一需求有共同基准

看架构压力

3

维护交接

未来维护者能理解当初为什么这样做

看设计边界

概念框架

产品意识：程序员也要理解业务、用户和数据

工程师的价值不只在实现功能，也在帮助团队做更好的产品判断。

1

商业维度

这个功能是否值得做

看业务动作

2

用户维度

它是否真的解决用户问题

看架构压力

3

数据维度

能否用数据验证效果

看设计边界

方法拆解

QAS：把质量属性写成可设计、可测试的场景

QAS 的价值是把“要快、要安全、要好维护”变成可工程化约束。

模糊说法

系统响应要快

→

工程写法

95% 预约提交请求在 2 秒内返回结果

→

架构意义

接口设计、缓存、数据库查询和测试目标都有了依据

转译路径的关键不是把字段列全，而是让每一步都能产生可检查的工程制品。

QAS 六要素模板

一个合格的 QAS 必须说明刺激从哪里来、作用到哪里、系统如何响应以及如何度量。

1. 刺激来源
谁触发

→

2. 刺激
发生了什么

→

3. 制品
影响哪个模块/接口/数据

从触发到度量，形成可测试路径

4. 环境
系统处于什么状态

→

5. 响应
系统应该怎么做

→

6. 响应度量
用数字说明成功标准

考试锚点 **综合题：按六要素写 QAS。**

方法拆解

QAS 示例一：安全性

安全需求必须写清攻击或越权场景，而不是只写“系统要安全”。

刺激来源

来自校外网络的未认证访问者

刺激

直接访问预约接口

制品

认证模块与预约接口

环境

系统正常运行

响应

拒绝请求，不暴露内部信息

响应度量

100% 未认证访问被拦截

响应度量必须可验证，否则 QAS 仍然只是愿望。

考试锚点 **考试可要求写安全性 QAS。**

方法拆解

QAS 示例二：可维护性

可维护性需求要面向未来接手者，而不是只面向当前开发团队。

刺激来源

学院 IT 工程师

刺激

修改设备分类业务逻辑

制品

设备管理模块

环境

上线 6 个月后，原团队离校

响应

能理解结构并独立修改

响应度量

平均修改不超过 1 天，有文档支撑

响应度量必须可验证，否则 QAS 仍然只是愿望。

考试锚点　**考试可问：可维护性如何量化？**

QAS 与架构决策的连接

QAS 写清楚后，架构师才知道该保护什么、取舍什么。

安全性 QAS

认证、授权、错误信息隐藏、日志审计

质量属性

→

可维护性 QAS

模块边界、文档、低复杂度技术栈

架构策略

→

性能 QAS

缓存、索引、异步处理、容量测试

验证方式

考试锚点 综合题：说明 QAS 如何影响架构选择。

练习/考试

课堂练习：诊断一份需求文档

需求文档诊断要先看三类需求是否完整，而不是只数功能条目。

文档现状

只有注册、登录、查看设备、预约、审批、通知

先诊断文档

第一步

诊断缺少哪些需求类型

第二步

补写 2 条 QAS

第三步

识别至少 4 类利益相关者

课堂动作 8 分钟课堂练习。

练习/考试

练习第一步：三类需求诊断

缺失质量属性和约束，是学生需求文档最常见的问题。

类型	文档中已有	缺口判断
功能需求	已有 6 条，但缺少场景和异常	需要补充可验证标准
质量属性需求	几乎没有：性能、安全、可维护、可用性未写	需要补充可验证标准
约束条件	缺少部署环境、团队规模、技术栈、合规要求	需要补充可验证标准

课堂动作　**教师讲评：不要只说“功能不够”。**

练习/考试

练习第二步：补写 QAS

补写 QAS 时，响应度量必须量化。

安全性

未认证访问预约接口被拒绝，不暴露内部路径

补全六要素

可维护性

学院 IT 能在 1 天内修改设备分类逻辑

量化响应度量

书写要求：把“谁触发、发生什么、系统怎样响应、用什么指标验收”补完整。

课堂动作　**学生分享：响应度量是否可测试？**

练习/考试

练习第三步：识别利益相关者

遗漏利益相关者，会导致需求只满足一部分人的视角。

研究生

预约便捷、反馈及时

实验室管理员

审批清楚、减少沟通

学院 IT

可维护、技术栈熟悉

找冲突
再写需求

练习重点：找出两组冲突诉求

信息中心

统一认证、数据安全

指导教师

设备利用率和记录可见

开发团队

范围可控、按期交付

课堂动作 **考试：给案例识别利益相关者与诉求冲突。**

从需求摘要进入 UML 建模

Persona 和场景是下一讲用例图、类图和交互图的输入。

Persona

识别参与者和目标

→

核心场景

形成用例与主流程

→

异常场景

补充扩展流程

→

领域对象

沉淀类图候选对象

每个环节都要接收上一环节的工程制品，并为下一环节留下可检查输入。

从 QAS 进入架构设计

QAS 是第6讲架构决策的需求输入。

安全性

认证、授权、审计日志

可维护性

模块边界、文档、低复杂度技术栈

可用性

降级、备份、故障恢复

性能

索引、缓存、异步处理

考试锚点 考试：同功能不同 QAS 为什么架构不同?

练习/考试

M1 更新：需求摘要进入课程项目

M1 不是增加文档负担，而是为后续建模和架构设计准备输入。

Persona

2-3 个，包含目标、痛点、限制

核心场景

3-5 个，包含角色、触发、操作、结果

QAS 清单

至少 3 条，六要素完整且可量化

变更判断

列出 1 个骨架变更和 1 个皮肤变更

考试锚点 **课后任务锚点。**

练习/考试

考试锚点一：三类需求与利益相关者

考试不会只考背定义，更会考你能否在案例中识别缺失需求。

名词解释

三类需求、质量属性需求、约束条件

题型

简答题

识别利益相关者及诉求冲突

答题骨架

案例题

诊断需求文档缺少什么

常见失分点

考试锚点 复习时要能各举一个实验室系统例子。

练习/考试

考试锚点二：Persona、场景与原型

Persona 和场景考的是分析能力，不是写人物小说。

Persona

写清目标、痛点、限制

题型

场景

角色、背景、触发、操作、期望结果

答题骨架

原型

说明低/中/高保真分别验证什么

常见失分点

考试锚点 复习时不要把 Persona 写成用户类别。

练习/考试

考试锚点三：QAS 与架构驱动

QAS 是本讲最可能进入综合案例题的工具。

QAS 六要素

刺激来源、刺激、制品、环境、响应、响应度量

题型

关键要求

响应度量必须量化

答题骨架

架构连接

说明 QAS 如何影响架构策略

常见失分点

考试锚点 综合题：写至少 2 条 QAS 并说明架构影响。

方法拆解

三条必须带走的结论

需求工程的目标，是把模糊愿望变成可验证、可设计、可演进的工程输入。

1. 需求不是功能清单 还要包含质量属性、约束和利益相关者

2. Persona 与场景不是形式 它们把抽象用户变成可分析的使用情境

3. QAS 连接需求与架构 质量属性必须量化，才能驱动架构决策

结论 **结束提问：你的小组项目最缺哪一类需求?**

问题交流

围绕需求、场景、Persona、QAS 与课程项目进行讨论

Any Questions?

把问题具体到：用户是谁、场景是什么、质量目标如何量化、会影响哪些架构决策。